

NVIDIA Cumulus Workshop Lab Walkthrough Guide

NVIDIA Cumulus Linux Workshop

NVIDIA Cumulus Linux 5.16.1

May 2026

NVIDIA Cumulus Linux Workshop | Lab Walkthrough Guide

NVIDIA Cumulus Linux Workshop Lab Walkthrough Guide.....3

Login Credentials.....3

Lab 1: Verifying Lab Connectivity & Setup.....3

Lab 2: Interface Configuration6

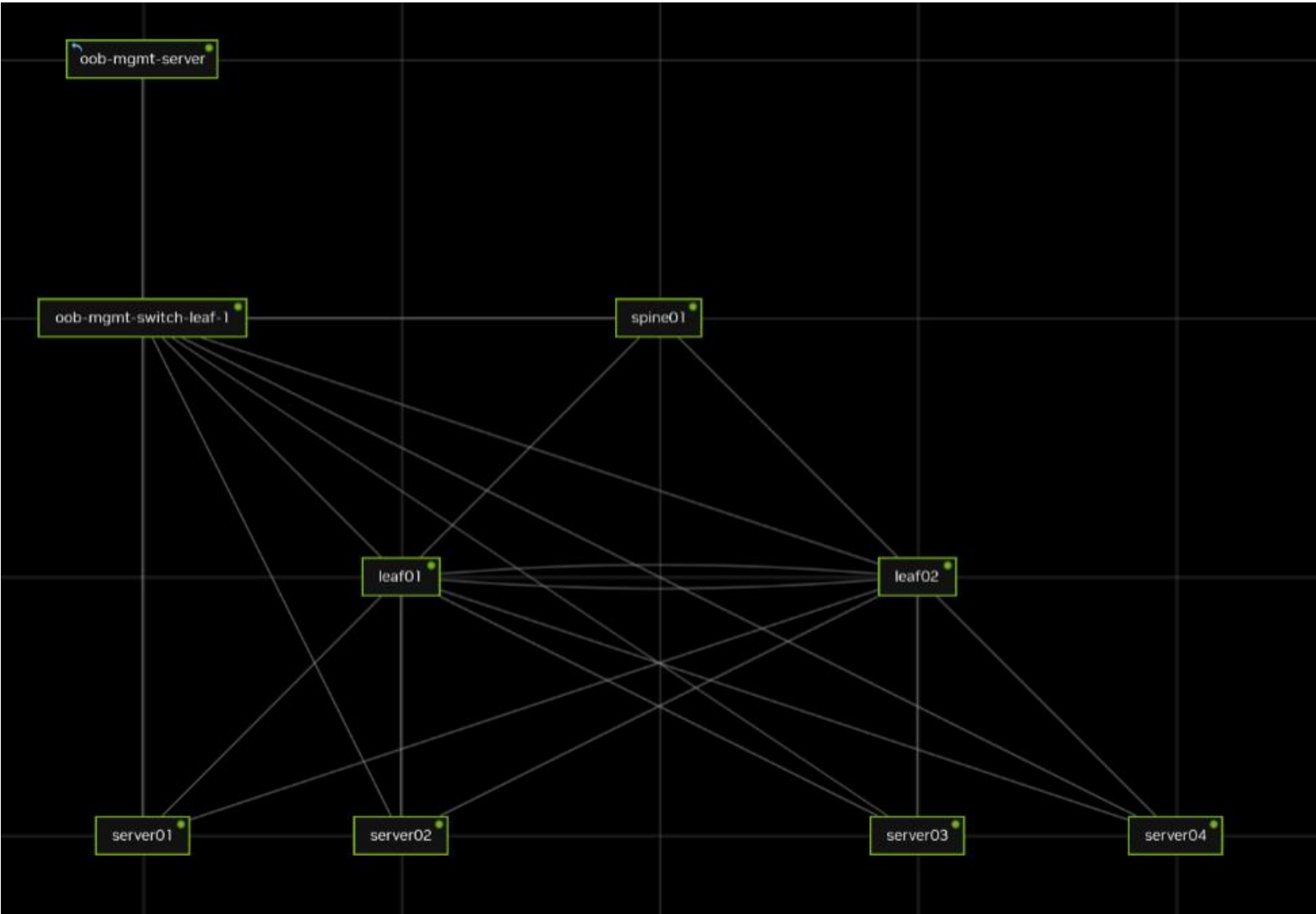
Lab 3: BGP Unnumbered and VRF-lite Configuration..... 11

Appendix A: Enable SSH In Your Lab..... 17

NVIDIA Cumulus Linux Workshop Lab Walkthrough Guide

This document will guide you through basic network configuration on Cumulus Linux within NVIDIA Air.

The lab runs the following topology:



Login Credentials

Login Credentials			
	leaf01 / leaf02 / spine01	servers	oob-mgmt-server
Username	cumulus	ubuntu	ubuntu
Password	Cumu1usLinux!	nvidia	nvidia

Lab 1: Verifying Lab Connectivity & Setup

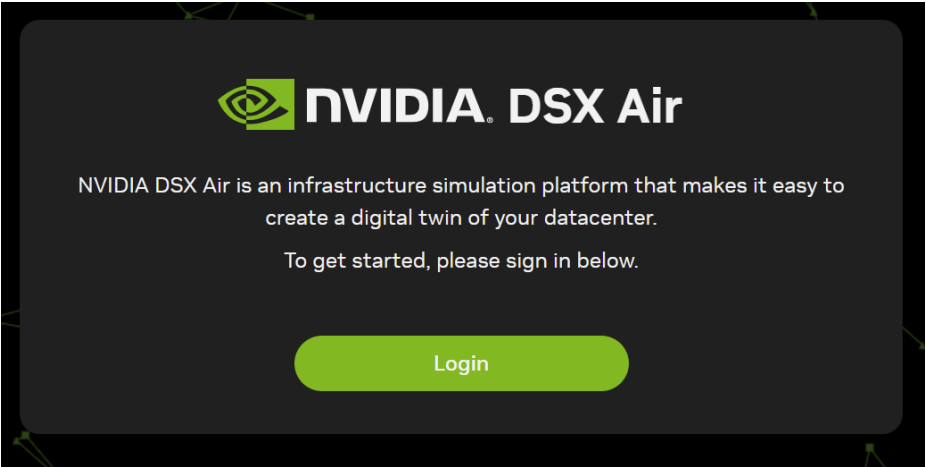
In this lab, let’s confirm you can access your NVIDIA Air lab/simulation. This involves first visiting your Air lab in a web browser. You will connect to your lab via SSH to an out-of-band management server (`oob-mgmt-server`).

- > Learn how to access your Air lab in a web browser.
- > Log into your `oob-mgmt-server`.
- > From your `oob-mgmt-server`, run the setup Ansible playbooks.

Access NVIDIA AIR

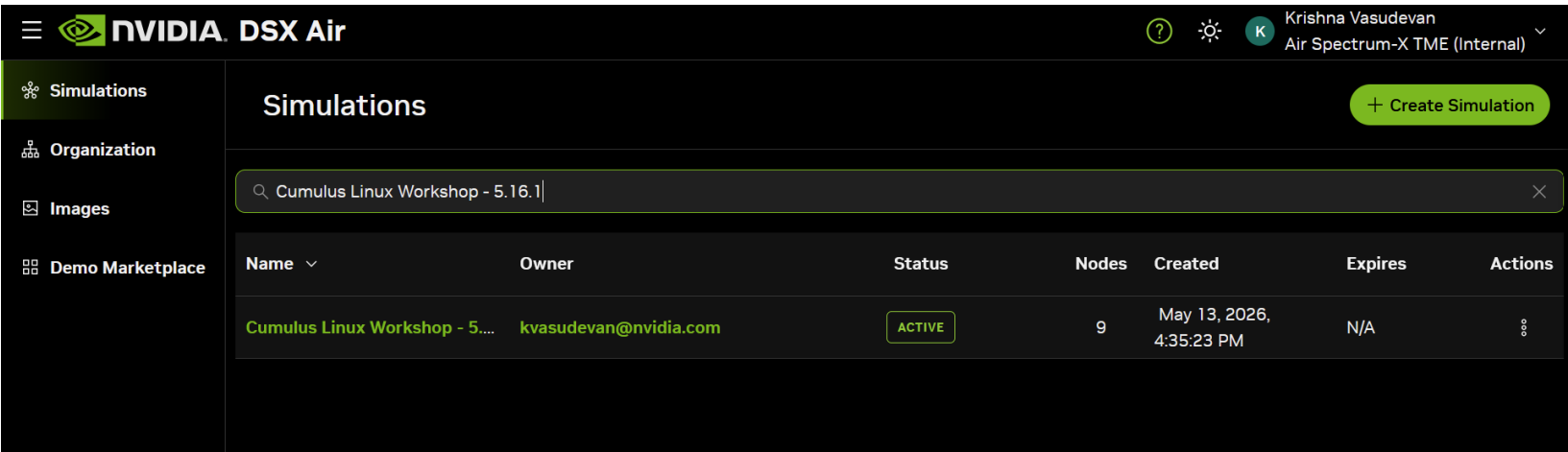
1. Log into <https://air-ngc.nvidia.com/>. Use the same email address you used to sign up for the workshop.

If you have not already, you must create an account. To do so, enter your business email and follow the prompts to create a new Air account. *New Air accounts must use a business email address. Non-business emails such as gmail.com or yahoo.com will not be accepted.*

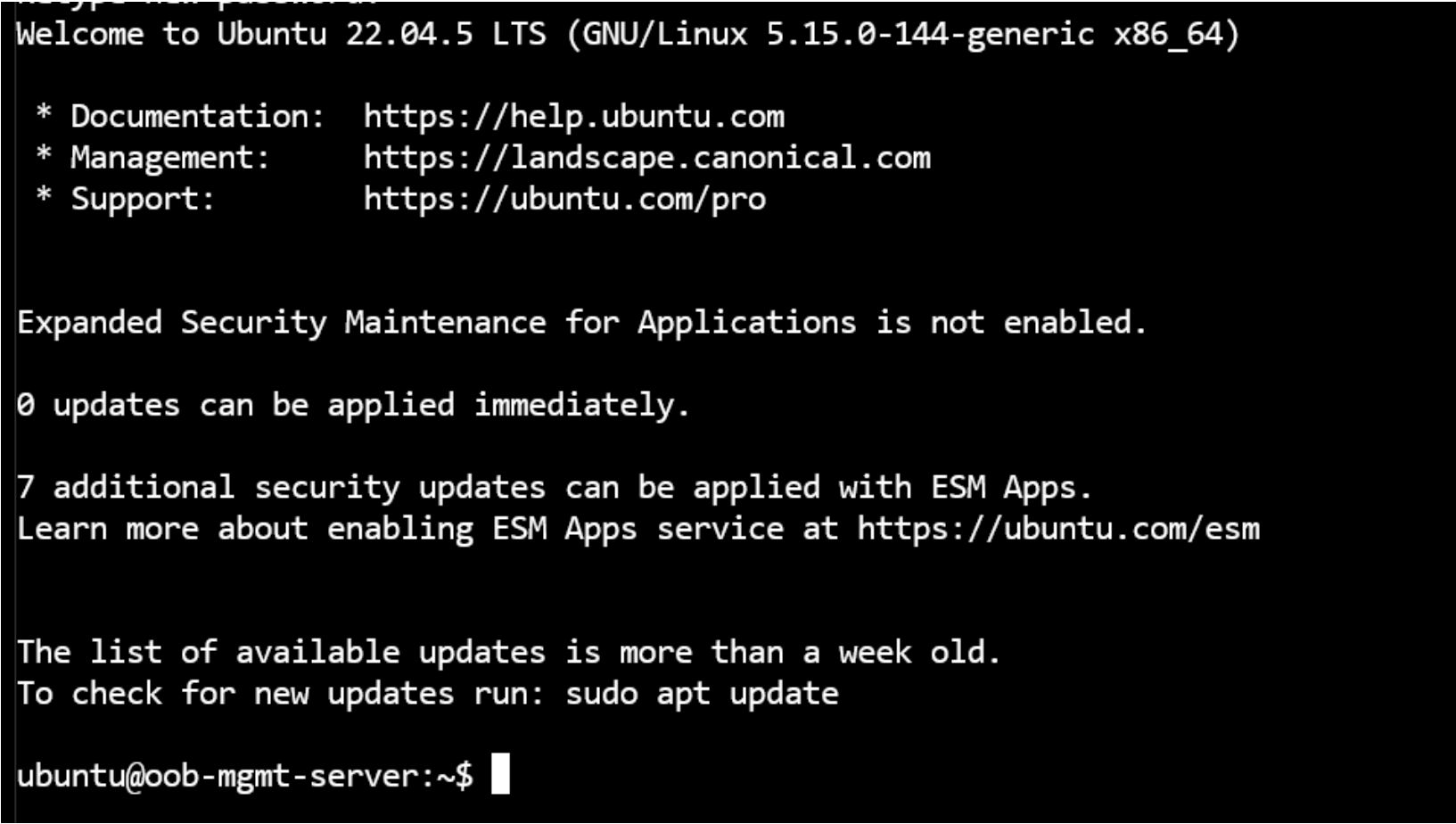


- Click the **Cumulus Linux Workshop – 5.16.1** simulation.

If you do not see this sim, it has either not been provisioned yet, as it is not close enough in time to the event, or you have used a different email to log into Air than the email we have on file for you.



- Double click on a node to open its console. You can drag and drop nodes away from each other if they appear stacked.
- Double click the `oob-mgmt-server` node. Login with the credentials found in the [Login Credentials](#) table.



Run the Setup Playbook

We must run a playbook to set the lab up for us. They have already been cloned into your lab. **You can find the GitLab repository [here](#).** On GitLab, you can also view extra documentation, bonus Ansible scripts, the topology used for the lab and more.

- In the `oob-mgmt-server`, change to the `Test-Drive-Automation` directory.

```
ubuntu@oob-mgmt-server:~$ cd Test-Drive-Automation
ubuntu@oob-mgmt-server:~/Test-Drive-Automation$
```

- The GitLab repository has already been cloned for you in the `Test-Drive-Automation` directory. Perform a `git pull` to sync/fetch changes.

```
ubuntu@oob-mgmt-server:~/Test-Drive-Automation$ git pull
Already up to date.
ubuntu@oob-mgmt-server:~/Test-Drive-Automation$
```

- Run the `start-lab.yml` Ansible playbook. This playbook configures the Ubuntu servers, including assigning an IP address, and explicitly sets the hostname on each leaf and spine switch.

```
ansible-playbook start-lab.yml

ubuntu@oob-mgmt-server:~/Test-Drive-Automation$ ansible-playbook start-lab.yml -i inventories/hosts
[WARNING]: Invalid characters were found in group names but not replaced, use -vvvv to see details

PLAY [leaf:spine]
*****
*****
```

```
TASK [Drop the saved NVUE yaml file]
*****
*****
Tuesday 09 September 2025  11:03:47 +0000 (0:00:00.038)          0:00:00.038 *****
changed: [spine01]
changed: [leaf01]
changed: [leaf02]

RUNNING HANDLER [nvue config patch]
*****
*****
Tuesday 09 September 2025  11:03:48 +0000 (0:00:01.673)          0:00:01.712 *****
changed: [leaf02]
changed: [leaf01]
changed: [spine01]

RUNNING HANDLER [nvue config apply]
*****
*****
Tuesday 09 September 2025  11:03:50 +0000 (0:00:01.341)          0:00:03.054 *****
changed: [spine01]
changed: [leaf02]
changed: [leaf01]

PLAY [host]
*****
*****

TASK [Setting up the test hosts config]
*****
*****
Tuesday 09 September 2025  11:03:57 +0000 (0:00:03.533)          0:00:10.029 *****
changed: [server03]
changed: [server02]
changed: [server01]
changed: [server04]

TASK [install traceroute]
*****
*****
Tuesday 09 September 2025  11:03:59 +0000 (0:00:02.783)          0:00:12.813 *****
changed: [server03]
changed: [server04]
changed: [server02]
changed: [server01]

TASK [flush arp]
*****
*****
Tuesday 09 September 2025  11:04:06 +0000 (0:00:06.658)          0:00:19.472 *****
changed: [server01]
changed: [server04]
changed: [server02]
changed: [server03]

RUNNING HANDLER [apply interface config]
*****
*****
Tuesday 09 September 2025  11:04:07 +0000 (0:00:01.370)          0:00:20.842 *****
changed: [server02]
changed: [server01]
changed: [server03]
changed: [server04]

PLAY RECAP
*****
*****
leaf01                : ok=4    changed=4    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
leaf02                : ok=4    changed=4    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
server01               : ok=4    changed=4    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
server02               : ok=4    changed=4    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
server03               : ok=4    changed=4    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
server04               : ok=4    changed=4    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
```

```
spine01          : ok=4   changed=4   unreachable=0   failed=0   skipped=0   rescued=0
ignored=0

Tuesday 09 September 2025  11:04:11 +0000 (0:00:03.461)      0:00:24.303 *****
=====
install traceroute -----
----- 6.66s
netq config restart agent -----
----- 3.53s
apply interface config -----
----- 3.46s
nvue config apply -----
----- 3.44s
Setting up the test hosts config -----
----- 2.78s
Drop the saved NVUE yaml file -----
----- 1.67s
flush arp -----
----- 1.37s
nvue config patch -----
----- 1.34s
ubuntu@oob-mgmt-server:~/Test-Drive-Automation$
```

Lab 2: Interface Configuration

This lab will configure interfaces to establish connection between the servers.

We will configure a bond between **leaf01** and **leaf02** as a trunk to pass **vlan10** and **vlan20**. Connections between leaf switches and servers will be configured as access ports. **Server01** and **Server02** will be in different subnets, so **leaf01** and **leaf02** will be configured to route for each VLAN using VRR to provide high availability gateways for each VLAN.

- > Configure loopbacks, bonds and bridges
- > Create trunk and access port
- > Configure SVIs and VRR addresses on **leaf01** and **leaf02**

Configure Loopbacks

Let’s configure loopbacks for our spine and leaf switches. Loopbacks serve as stable endpoints for BGP peering.

Loopbacks			
	leaf01	leaf02	spine01
Loopback IP	10.255.255.1/32	10.255.255.2/32	10.255.255.101/32

1. **[leaf01]:** On **leaf01**, assign **10.255.255.1/32** to **lo**. Apply to the running configuration. Verify. *The entire output is shown here for **leaf01**, but show commands are condensed for all further verification for easier viewing.*

```
cumulus@leaf01:mgmt:~$ nv set interface lo ipv4 address 10.255.255.1/32
created [rev_id: 6]
cumulus@leaf01:mgmt:~$ nv config apply
applied_and_saved [rev_id: 6]
```

```
cumulus@leaf01:mgmt:~$ nv show interface lo
operational      applied
-----
type             loopback        loopback
router
  ospf
    state         disabled
  pim
    state         disabled
  ospf6
    state         disabled
ptp
  state           disabled
neighbor
  [ipv4]
  [ipv6]
sflow
  state           enabled
ipv4
  forward         enabled
  igmp
    state         disabled
  dhcp-client
    state         disabled
    set-hostname disabled
  [address]       10.255.255.1/32  10.255.255.1/32
  [address]       127.0.0.1/8
```



```
ipv6
  forward                                enabled
  dhcp-client
    state                                disabled
    set-hostname                        disabled
  [address]                             ::1/128
  state                                  enabled
link
  mac-address                           00:00:00:00:00:00
  mtu                                    65536
  [flag]                                loopback
  [flag]                                up
  [flag]                                lower-up
  state                                  up
  protodown                             disabled
  oper-status                           unknown
  admin-status                          up
  oper-status-last-change               Never
counters
  link
    carrier-transitions                 0
    carrier-up-count                   0
    carrier-down-count                 0
ifindex                                1
```

2. **[leaf02]:** On **leaf02**, assign **10.255.255.2/32** to **lo**. Apply to the running configuration. Verify.

```
cumulus@leaf02:mgmt:~$ nv set interface lo ipv4 address 10.255.255.2/32
created [rev_id: 6]
cumulus@leaf02:mgmt:~$ nv config apply
applied_and_saved [rev_id: 6]
```

```
cumulus@leaf02:mgmt:~$ nv show interface lo ipv4 address

IPv4 Address      Address type
-----
10.255.255.2/32   primary
127.0.0.1/8       primary
```

3. **[spine01]:** On **spine01**, assign **10.255.255.101/32** to **lo**. Apply to the running configuration. Verify.

```
cumulus@spine01:mgmt:~$ nv set interface lo ipv4 address 10.255.255.101/32
created [rev_id: 6]
cumulus@spine01:mgmt:~$ nv config apply
applied_and_saved [rev_id: 6]
```

```
cumulus@spine01:mgmt:~$ nv show interface lo ipv4 address

IPv4 Address      Address type
-----
10.255.255.101/32 primary
127.0.0.1/8       primary
```

Configure Bonds

Let's configure the bond between **leaf01** and **leaf02**.

1. **[leaf01]:** On **leaf01**, create bond **bond0** with members **swp49** and **swp50**. Apply to the running configuration. Verify.

```
cumulus@leaf01:mgmt:~$ nv set interface bond0 bond member swp49-50
cumulus@leaf01:mgmt:~$ nv config apply
```

```
cumulus@leaf01:mgmt:~$ nv show interface bond0 bond member
      bonding-state  mii-status
-----
swp49  backup       up
swp50  backup       up
```

```
cumulus@leaf01:mgmt:~$ nv show interface bond0 link state
operational  applied
-----
up           up
```

2. **[leaf02]:** On **leaf02**, create bond **bond0** with members **swp49** and **swp50**. Apply to the running configuration. Verify.

```
cumulus@leaf02:mgmt:~$ nv set interface bond0 bond member swp49-50
cumulus@leaf02:mgmt:~$ nv config apply
```

```
cumulus@leaf02:mgmt:~$ nv show interface bond0 bond member
      bonding-state  mii-status
```

```
-----
swp49  active      up
swp50  active      up

cumulus@leaf02:mgmt:~$ nv show interface bond0 link state
operational  applied
-----
up           up
```

Configure Bridge and Access Ports

Let’s configure the bridge and access ports for **leaf01** and **leaf02**.

Bridge Configuration		
	leaf01	leaf02
VLANs	10, 20	10, 20
Members	bond0, swp1	bond0, swp2
Access Port	swp1	swp2
Access VLAN	10	20

1. **[leaf01]:** On **leaf01**, create a **bridge** with **VLANs 10** and **20**.

```
cumulus@leaf01:mgmt:~$ nv set bridge domain br_default vlan 10,20
```

2. **[leaf01]:** Add **swp1** and **bond0** as members to the bridge.

```
cumulus@leaf01:mgmt:~$ nv set interface swp1,bond0 bridge domain br_default
```

3. **[leaf01]:** Add **swp1** as an access port for **VLAN10**. Apply to the running configuration. Verify.

```
cumulus@leaf01:mgmt:~$ nv set interface swp1 bridge domain br_default access 10
cumulus@leaf01:mgmt:~$ nv config apply
```

```
cumulus@leaf01:mgmt:~$ nv show bridge domain br_default vlan

Vlan  Ptp State  Source IP  VNI
----  -
10    disabled  0.0.0.0
20    disabled  0.0.0.0
```

4. **[leaf02]:** Repeat for **leaf02** with **swp2** and access port for **VLAN20**. A code snippet is provided below for easy copy and paste.

```
nv set bridge domain br_default vlan 10,20
nv set interface swp2,bond0 bridge domain br_default
nv set interface swp2 bridge domain br_default access 20
nv config apply
```

Configure SVI and VRR

Let’s configure the switch virtual interface (SVI) and virtual router redundancy (VRR) for **leaf01** and **leaf02**.

VRR Configuration		
	leaf01	leaf02
VLAN10 real IP	10.0.10.2/24	10.0.10.3/24
VLAN10 VRR IP	10.0.10.1/24	10.0.10.1/24
VLAN10 VRR MAC	00:00:00:00:1a:10	00:00:00:00:1a:10
VLAN20 real IP	10.0.20.2/24	10.0.20.3/24
VLAN20 VRR IP	10.0.20.1/24	10.0.20.1/24
VLAN20 VRR MAC	00:00:00:00:1a:20	00:00:00:00:1a:20
Server01 VLAN	10	10
Server02 VLAN	20	20

1. **[leaf01]:** On **leaf01**, create an SVI for **VLAN10**.


```
cumulus@leaf01:mgmt:~$ nv set interface vlan10 ipv4 address 10.0.10.2/24
```

2. **[leaf01]:** Create an SVI for **VLAN20**.

```
cumulus@leaf01:mgmt:~$ nv set interface vlan20 ipv4 address 10.0.20.2/24
```

3. **[leaf01]:** Apply a VRR address for **VLAN10**.

```
cumulus@leaf01:mgmt:~$ nv set interface vlan10 ipv4 vrr address 10.0.10.1/24
cumulus@leaf01:mgmt:~$ nv set interface vlan10 ipv4 vrr mac-address 00:00:00:00:1a:10
cumulus@leaf01:mgmt:~$ nv set interface vlan10 ipv4 vrr state enabled
```

4. **[leaf01]:** Apply a **VRR** address for **VLAN20**. Apply to the running configuration.

```
cumulus@leaf01:mgmt:~$ nv set interface vlan20 ipv4 vrr address 10.0.20.1/24
cumulus@leaf01:mgmt:~$ nv set interface vlan20 ipv4 vrr mac-address 00:00:00:00:1a:20
cumulus@leaf01:mgmt:~$ nv set interface vlan20 ipv4 vrr state enabled
cumulus@leaf01:mgmt:~$ nv config apply
```

5. **[leaf02]:** Repeat for **leaf02**. A code snippet is provided below for easy copy and paste.

```
nv set interface vlan10 ipv4 address 10.0.10.3/24
nv set interface vlan10 ipv4 vrr address 10.0.10.1/24
nv set interface vlan10 ipv4 vrr mac-address 00:00:00:00:1a:10
nv set interface vlan10 ipv4 vrr state enabled
nv set interface vlan20 ipv4 address 10.0.20.3/24
nv set interface vlan20 ipv4 vrr address 10.0.20.1/24
nv set interface vlan20 ipv4 vrr mac-address 00:00:00:00:1a:20
nv set interface vlan20 ipv4 vrr state enabled
nv config apply
```

Test VRR Connectivity

Let's verify our VRR gateways are working by testing server connectivity.

1. **[server01]:** On **server01**, ping the **VLAN10 VRR** address **10.0.10.1** for **leaf01**.

```
ubuntu@server01:~$ ping 10.0.10.1
PING 10.0.10.1 (10.0.10.1) 56(84) bytes of data.
64 bytes from 10.0.10.1: icmp_seq=1 ttl=64 time=0.686 ms
64 bytes from 10.0.10.1: icmp_seq=2 ttl=64 time=0.922 ms
^C
--- 10.0.10.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.686/0.804/0.922/0.118 ms
```

2. **[server01]:** Ping the **VLAN10 real** address **10.0.10.2** for **leaf01**.

```
ubuntu@server01:~$ ping 10.0.10.2
PING 10.0.10.2 (10.0.10.2) 56(84) bytes of data.
64 bytes from 10.0.10.2: icmp_seq=1 ttl=64 time=0.887 ms
64 bytes from 10.0.10.2: icmp_seq=2 ttl=64 time=0.835 ms
^C
--- 10.0.10.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.835/0.861/0.887/0.026 ms
```

3. **[server01]:** Ping the **VLAN10 real** address **10.0.10.3** for **leaf02**.

```
ubuntu@server01:~$ ping 10.0.10.3
PING 10.0.10.3 (10.0.10.3) 56(84) bytes of data.
64 bytes from 10.0.10.3: icmp_seq=1 ttl=64 time=0.528 ms
64 bytes from 10.0.10.3: icmp_seq=2 ttl=64 time=0.876 ms
^C
--- 10.0.10.3 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.528/0.702/0.876/0.174 ms
```

4. **[server01]:** Check the IP neighbor table. Observe that each address is **REACHABLE** or **STALE**.

```
ubuntu@server01:~$ ip neighbor show
192.168.200.254 dev eth0 lladdr 48:c2:b0:2d:01:01 STALE
192.168.200.1 dev eth0 lladdr 48:b0:2d:00:00:01 STALE
10.0.10.3 dev eth1 lladdr 44:38:39:22:01:7d REACHABLE
10.0.10.2 dev eth1 lladdr 44:38:39:22:01:b1 REACHABLE
10.0.10.1 dev eth1 lladdr 00:00:00:00:1a:10 STALE
fe80::4ac2:b0ff:fe2d:101 dev eth0 lladdr 48:c2:b0:2d:01:01 router STALE
```

5. **[server01]:** Ping **server02**.

```
ubuntu@server01:~$ ping 10.0.20.102
PING 10.0.20.102 (10.0.20.102) 56(84) bytes of data.
64 bytes from 10.0.20.102: icmp_seq=1 ttl=63 time=0.790 ms
64 bytes from 10.0.20.102: icmp_seq=2 ttl=63 time=1.35 ms
^C
--- 10.0.20.102 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.790/1.070/1.351/0.282 ms
```

6. **[server01]:** Traceroute **server02**.

```
ubuntu@server01:~$ traceroute 10.0.20.102
traceroute to 10.0.20.102 (10.0.20.102), 30 hops max, 60 byte packets
 1  10.0.10.1 (10.0.10.1)  1.628 ms  1.672 ms  1.855 ms
 2  10.0.20.102 (10.0.20.102)  7.947 ms  7.973 ms  8.155 ms
```

7. **[server02]:** On **server02**, ping the **VLAN20 VRR** address **10.0.20.1** for **leaf01**.

```
ubuntu@server02:~$ ping 10.0.20.1
PING 10.0.20.1 (10.0.20.1) 56(84) bytes of data.
64 bytes from 10.0.20.1: icmp_seq=1 ttl=64 time=1.22 ms
64 bytes from 10.0.20.1: icmp_seq=2 ttl=64 time=0.672 ms
^C
--- 10.0.20.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.672/0.949/1.226/0.277 ms
```

8. **[server02]:** Ping the **VLAN20 real** address **10.0.20.2** for **leaf01**.

```
ubuntu@server02:~$ ping 10.0.20.2
PING 10.0.20.2 (10.0.20.2) 56(84) bytes of data.
64 bytes from 10.0.20.2: icmp_seq=1 ttl=64 time=0.735 ms
64 bytes from 10.0.20.2: icmp_seq=2 ttl=64 time=1.02 ms
^C
--- 10.0.20.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.735/0.882/1.029/0.147 ms
```

9. **[server02]:** Ping the **VLAN20 real** address **10.0.20.3** for **leaf02**.

```
ubuntu@server02:~$ ping 10.0.20.3
PING 10.0.20.3 (10.0.20.3) 56(84) bytes of data.
64 bytes from 10.0.20.3: icmp_seq=1 ttl=64 time=0.993 ms
64 bytes from 10.0.20.3: icmp_seq=2 ttl=64 time=1.08 ms
^C
--- 10.0.20.3 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 0.993/1.040/1.087/0.047 ms
```

10. **[server02]:** Check the IP neighbor table. Observe that each address is **REACHABLE** or **STALE**.

```
ubuntu@server02:~$ ip neighbor show
10.0.20.3 dev eth2 lladdr 44:38:39:22:01:7d REACHABLE
10.0.20.2 dev eth2 lladdr 44:38:39:22:01:b1 REACHABLE
192.168.200.254 dev eth0 lladdr 48:c2:b0:2d:01:01 STALE
10.0.20.1 dev eth2 lladdr 00:00:00:00:1a:20 REACHABLE
192.168.200.1 dev eth0 lladdr 48:b0:2d:00:00:01 STALE
fe80::4ac2:b0ff:fe2d:101 dev eth0 lladdr 48:c2:b0:2d:01:01 router STALE
```

11. **[server02]:** Ping **server01**.

```
ubuntu@server02:~$ ping 10.0.10.101
PING 10.0.10.101 (10.0.10.101) 56(84) bytes of data.
64 bytes from 10.0.10.101: icmp_seq=1 ttl=63 time=1.08 ms
64 bytes from 10.0.10.101: icmp_seq=2 ttl=63 time=1.36 ms
^C
--- 10.0.10.101 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 1.089/1.225/1.361/0.136 ms
```

12. **[server02]:** Traceroute **server01**.

```
ubuntu@server02:~$ traceroute 10.0.10.101
traceroute to 10.0.10.101 (10.0.10.101), 30 hops max, 60 byte packets
 1  10.0.20.1 (10.0.20.1)  2.813 ms  2.776 ms  3.307 ms
 2  10.0.10.101 (10.0.10.101)  9.199 ms  7.836 ms  7.766 ms
```

Verify Learned MAC Addresses

Verify the learned MAC addresses match.

1. **[leaf01]:** Observe learned MACs.

```
cumulus@leaf01:mgmt:~$ nv show bridge domain br_default mac-table
```

entry-id	MAC address	vlan	interface	remote-dst	src-vni	entry-type	last-update	age
1	48:b0:2d:15:38:8c	10	swp1				0:03:30	0:00:26
2	48:b0:2d:59:a2:0b		swp1			permanent	5:54:20	5:54:20
3	48:b0:2d:0d:5a:32	20	bond0				5:49:10	0:00:02
4	44:38:39:22:01:7d	20	bond0				5:53:03	0:00:53
5	44:38:39:22:01:7d	10	bond0				0:01:47	0:00:26
6	48:b0:2d:1f:56:b3	1	bond0				5:54:13	0:00:10
7	48:b0:2d:79:d6:4e	1	bond0				5:54:13	0:00:10
8	48:b0:2d:96:44:cd	1	bond0			permanent	5:54:20	5:54:20
9	48:b0:2d:96:44:cd		bond0			permanent	5:54:20	5:54:20
10	00:00:00:00:1a:10		br_default			permanent		
11	00:00:00:00:1a:20		br_default			permanent		
12	00:00:00:00:1a:20	20	br_default			permanent	5:53:12	5:53:12
13	44:38:39:22:01:7c	20	br_default			permanent	5:53:12	5:53:12
14	00:00:00:00:1a:10	10	br_default			permanent	5:53:12	5:53:12
15	44:38:39:22:01:7c	10	br_default			permanent	5:53:12	5:53:12
16	44:38:39:22:01:7c	1	br_default			permanent	5:54:20	5:54:20
17	44:38:39:22:01:7c		br_default			permanent	5:54:20	5:54:20

2. **[leaf02]:** Observe learned MACs.

```
cumulus@leaf02:mgmt:~$ nv show bridge domain br_default mac-table
```

entry-id	MAC address	vlan	interface	remote-dst	src-vni	entry-type	last-update	age
1	48:b0:2d:0d:5a:32	20	swp2				0:01:11	0:00:05
2	48:b0:2d:75:89:84		swp2			permanent	5:57:42	5:57:42
3	48:b0:2d:15:38:8c	10	bond0				5:56:06	0:00:12
4	44:38:39:22:01:7c	20	bond0				0:00:22	0:00:22
5	44:38:39:22:01:7c	10	bond0				5:56:44	0:00:49
6	48:b0:2d:25:cd:6c	1	bond0				5:57:15	0:00:12
7	48:b0:2d:96:44:cd	1	bond0				5:57:40	0:00:00
8	48:b0:2d:1f:56:b3	1	bond0			permanent	5:57:42	5:57:42
9	48:b0:2d:1f:56:b3		bond0			permanent	5:57:42	5:57:42
10	00:00:00:00:1a:10		br_default			permanent		
11	00:00:00:00:1a:20		br_default			permanent		
12	44:38:39:22:01:7d	20	br_default			permanent	5:56:35	5:56:35
13	44:38:39:22:01:7d	10	br_default			permanent	5:56:35	5:56:35
14	00:00:00:00:1a:20	20	br_default			permanent	5:56:35	5:56:35
15	00:00:00:00:1a:10	10	br_default			permanent	5:56:35	5:56:35
16	44:38:39:22:01:7d	1	br_default			permanent	5:57:42	5:57:42
17	44:38:39:22:01:7d		br_default			permanent	5:57:42	5:57:42

Lab 3: BGP Unnumbered and VRF-lite Configuration

Let’s configure unnumbered BGP and VRF-lite.

This configuration shares the loopback 10 IPs on each leaf and spine. It also shares the **VLAN10** and **VLAN20** subnets on **leaf01** and **leaf02**.

We will also configure VRF-lite in combination with BGP peering to carry VRF-specific prefixes.

- > Configure and verify BGP unnumbered on each spine and leaf
- > Configure and verify BGP peering between VRF member sub interfaces

Run Setup Playbook

1. **[oob-mgmt-server]:** On the **oob-mgmt-server**, run the setup playbook **lab3.yml**.

```
ubuntu@oob-mgmt-server:~/Test-Drive-Automation$ ansible-playbook lab3.yml -i inventories/hosts

[WARNING]: Invalid characters were found in group names but not replaced, use -vvvv to see details

PLAY [Getting you ready for lab3] *****

TASK [Dropping in config] *****
Wednesday 11 May 2022  17:32:27 +0000 (0:00:00.018)          0:00:00.018 *****
ok: [server02]
ok: [server01]

PLAY [Getting you ready for lab3] *****

TASK [Drop the nvue yaml] *****
Wednesday 11 May 2022  17:32:28 +0000 (0:00:01.362)          0:00:01.381 *****
changed: [leaf01]
changed: [leaf02]

RUNNING HANDLER [nvue config replace] *****
Wednesday 11 May 2022  17:32:29 +0000 (0:00:01.063)          0:00:02.445 *****
changed: [leaf02]
changed: [leaf01]

RUNNING HANDLER [nvue config apply] *****
Wednesday 11 May 2022  17:32:30 +0000 (0:00:01.088)          0:00:03.533 *****
changed: [leaf02]
```

```
changed: [leaf01]

PLAY RECAP *****
leaf01      : ok=3    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
leaf02      : ok=3    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
server01    : ok=1    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
server02    : ok=1    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

Wednesday 11 May 2022  17:32:35 +0000 (0:00:05.243)          0:00:08.777 *****
=====
nvue config apply ----- 5.24s
Dropping in config ----- 1.36s
nvue config replace ----- 1.09s
Drop the nvue yaml ----- 1.06s
```

Configure BGP Unnumbered

Let’s configure BGP unnumbered on **leaf01**, **leaf02** and **spine01**.

- 1. **[spine01]:** On **spine01**, configure a BGP Autonomous System (AS).

```
cumulus@spine01:mgmt:~$ nv set vrf default router bgp autonomous-system 65201
cumulus@spine01:mgmt:~$ nv set vrf default router bgp path-selection multipath aspath-ignore enabled
cumulus@spine01:mgmt:~$ nv set router bgp router-id 10.255.255.101
```

- 2. **[spine01]:** Configure BGP peering on **swp1** to **leaf01** and **swp2** to **leaf02**. Apply to the running configuration.

```
cumulus@spine01:mgmt:~$ nv set vrf default router bgp neighbor swp1 remote-as external
cumulus@spine01:mgmt:~$ nv set vrf default router bgp neighbor swp2 remote-as external
cumulus@spine01:mgmt:~$ nv set interface swp1 link state up
cumulus@spine01:mgmt:~$ nv set interface swp2 link state up
cumulus@spine01:mgmt:~$ nv config apply
Warning: The frr service will need to be restarted because the list of router services has
changed. This will disrupt traffic for any existing routing protocols.

Are you sure? [y/N] y
```

- 3. **[leaf01]:** Repeat for **leaf01**. A code snippet is provided below for easy copy and paste.

```
nv set vrf default router bgp autonomous-system 65101
nv set vrf default router bgp path-selection multipath aspath-ignore enabled
nv set vrf default router bgp router-id 10.255.255.1
nv set vrf default router bgp neighbor swp51 remote-as external
nv set interface swp51 link state up
nv config apply
Warning: The frr service will need to be restarted because the list of router services has changed.
This will disrupt traffic for any existing routing protocols.

Are you sure? [y/N] y
```

- 4. **[leaf02]:** Repeat for **leaf02**. A code snippet is provided below for easy copy and paste.

```
nv set vrf default router bgp autonomous-system 65102
nv set vrf default router bgp path-selection multipath aspath-ignore enabled
nv set vrf default router bgp router-id 10.255.255.2
nv set vrf default router bgp neighbor swp51 remote-as external
nv set interface swp51 link state up
nv config apply
Warning: The frr service will need to be restarted because the list of router services has changed.
This will disrupt traffic for any existing routing protocols.

Are you sure? [y/N] y
```

- 5. **[spine01]:** Verify BGP peering.

```
cumulus@spine01:mgmt:~$ sudo vtysh -c "show ip bgp summary"

IPv4 Unicast Summary:
BGP router identifier 10.255.255.101, local AS number 65201 VRF default vrf-id 0
BGP table version 0
RIB entries 0, using 0 bytes of memory
Peers 2, using 44 KiB of memory

Neighbor      V      AS  MsgRcvd  MsgSent  TblVer  InQ  OutQ  Up/Down  State/PfxRcd  PfxSnt  Desc
leaf01(swp1)  4      65101     31      32        0    0    0 00:01:25        0        0
FRRouting/10.0.3
leaf02(swp2)  4      65102      7        7        0    0    0 00:00:14        0        0
FRRouting/10.0.3

Total number of neighbors 2
```

- 6. **[leaf01]:** Verify BGP peering.

```
cumulus@leaf01:mgmt:~$ sudo vtysh -c "show ip bgp summary"

IPv4 Unicast Summary:
BGP router identifier 10.255.255.1, local AS number 65101 VRF default vrf-id 0
BGP table version 0
RIB entries 0, using 0 bytes of memory
Peers 1, using 22 KiB of memory

Neighbor      V      AS  MsgRcvd  MsgSent  TblVer  InQ  OutQ  Up/Down  State/PfxRcd  PfxSnt  Desc
spine01(swp51) 4      65201    45      45        0     0     0  00:02:07        0          0
FRRouting/10.0.3

Total number of neighbors 1
```

7. **[leaf02]:** Verify BGP peering.

```
cumulus@leaf02:mgmt:~$ sudo vtysh -c "show ip bgp summary"

IPv4 Unicast Summary:
BGP router identifier 10.255.255.2, local AS number 65102 VRF default vrf-id 0
BGP table version 0
RIB entries 0, using 0 bytes of memory
Peers 1, using 22 KiB of memory

Neighbor      V      AS  MsgRcvd  MsgSent  TblVer  InQ  OutQ  Up/Down  State/PfxRcd  PfxSnt  Desc
spine01(swp51) 4      65201    31      32        0     0     0  00:01:25        0          0
FRRouting/10.0.3

Total number of neighbors 1
```

Advertise Loopback and SVI Subnets

Routing Advertisement			
	leaf01	leaf02	spine01
Advertised Subnets	10.255.255.1/32 10.0.10.0/24	10.255.255.2/32 10.0.20.0/24	10.255.255.101/32

1. **[spine01]:** On **spine01**, advertise the loopback address into BGP. Apply to the running configuration.

```
cumulus@spine01:mgmt:~$ nv set vrf default router bgp address-family ipv4-unicast network 10.255.255.101/32
cumulus@spine01:mgmt:~$ nv config apply
```

2. **[leaf01]:** On **leaf01**, advertise the loopback address and subnet **VLAN10** into BGP. Apply to the running configuration.

```
cumulus@leaf01:mgmt:~$ nv set vrf default router bgp address-family ipv4-unicast network 10.255.255.1/32
cumulus@leaf01:mgmt:~$ nv set vrf default router bgp address-family ipv4-unicast network 10.0.10.0/24
cumulus@leaf01:mgmt:~$ nv config apply
```

3. **[leaf02]:** On **leaf02**, advertise the loopback address and subnet **VLAN20** into BGP. Apply to the running configuration.

```
cumulus@leaf02:mgmt:~$ nv set vrf default router bgp address-family ipv4-unicast network 10.255.255.2/32
cumulus@leaf02:mgmt:~$ nv set vrf default router bgp address-family ipv4-unicast network 10.0.20.0/24
cumulus@leaf02:mgmt:~$ nv config apply
```

4. **[spine01]:** Verify.

```
cumulus@spine01:mgmt:~$ sudo vtysh -c "show ip bgp ipv4 unicast"

BGP table version is 5, local router ID is 10.255.255.101, vrf id 0
Default local pref 100, local AS 65201
Status codes:  s suppressed, d damped, h history, u unsorted, * valid, > best, = multipath, + multipath nhg,
               i internal, r RIB-failure, S Stale, R Removed
Nexthop codes: @NNN nexthop's vrf id, < announce-nh-self
Origin codes:  i - IGP, e - EGP, ? - incomplete
RPKI validation codes: V valid, I invalid, N Not found

      Network          Next Hop              Metric LocPrf Weight Path
*> 10.0.10.0/24        swp1                      0              0 65101 i
*> 10.0.20.0/24        swp2                      0              0 65102 i
*> 10.255.255.1/32     swp1                      0              0 65101 i
*> 10.255.255.2/32     swp2                      0              0 65102 i
*> 10.255.255.101/32
                        0.0.0.0(spine01)
                        0              32768 i

Displayed 5 routes and 5 total paths
```

Quick Verify Connectivity with Servers

Let’s quickly verify our BGP is working by checking server to server ping again.

1. **[server01]:** On **server01**, ping **server02**.

```
ubuntu@server01:~$ ping 10.0.20.102
PING 10.0.20.102 (10.0.20.102) 56(84) bytes of data.
64 bytes from 10.0.20.102: icmp_seq=1 ttl=61 time=9.86 ms
64 bytes from 10.0.20.102: icmp_seq=2 ttl=61 time=5.96 ms
64 bytes from 10.0.20.102: icmp_seq=3 ttl=61 time=5.80 ms
^C
--- 10.0.20.102 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 5.806/7.211/9.864/1.877 ms
```

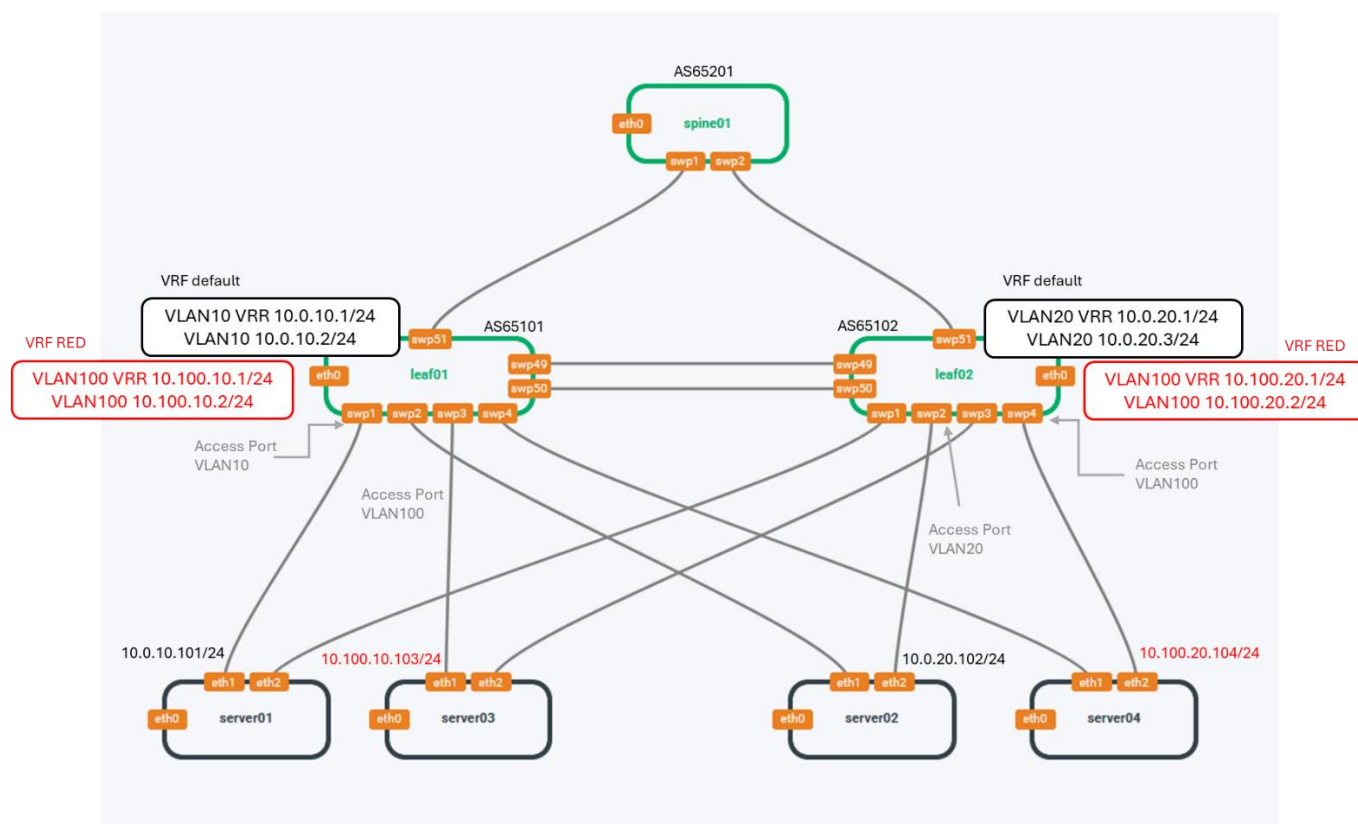
2. **[server01]:** Traceroute **server02**. Observe the hop to **spine01** and **leaf02** now present.

```
ubuntu@server01:~$ traceroute 10.0.20.102
traceroute to 10.0.20.102 (10.0.20.102), 30 hops max, 60 byte packets
1 10.0.10.1 (10.0.10.1) 1.280 ms 1.389 ms 1.553 ms
2 10.255.255.101 (10.255.255.101) 4.702 ms 4.679 ms 4.789 ms
3 10.255.255.2 (10.255.255.2) 8.438 ms 8.877 ms 9.476 ms
4 10.0.20.102 (10.0.20.102) 9.541 ms 9.766 ms 13.549 ms
```

VRF & Member Interfaces

Virtual routing and forwarding (VRF) enables you to use multiple independent routing tables that work simultaneously on the same switch. Other implementations call this feature **VRF-Lite (local)**.

Let's configure a VRF **RED** with the following topology. We will create a separate **VLAN100** and **SVI** for **VLAN100** mapped to the **RED** VRF. We will then use **server03** and **server04** to verify connectivity for **RED**.



1. **[leaf01]:** On **leaf01**, create a VRF **RED** and configure interfaces.

```
cumulus@leaf01:mgmt:~$ nv set vrf RED table auto
cumulus@leaf01:mgmt:~$ nv set bridge domain br_default vlan 100
cumulus@leaf01:mgmt:~$ nv set interface swp3 bridge domain br_default access 100
cumulus@leaf01:mgmt:~$ nv set interface vlan100 ipv4 vrr state enabled
cumulus@leaf01:mgmt:~$ nv set interface vlan100 type svi
cumulus@leaf01:mgmt:~$ nv set interface vlan100 ipv4 address 10.100.10.2/24
cumulus@leaf01:mgmt:~$ nv set interface vlan100 ipv4 vrr address 10.100.10.1/24
cumulus@leaf01:mgmt:~$ nv set interface vlan100 ipv4 vrr mac-address 00:00:00:00:2a:1
cumulus@leaf01:mgmt:~$ nv set interface swp51.100,vlan100 vrf RED
cumulus@leaf01:mgmt:~$ nv set interface swp51.100,vlan100 vlan 100
cumulus@leaf01:mgmt:~$ nv config apply
```

2. **[leaf02]:** Repeat for **leaf02**. A code snippet is provided below for easy copy and paste.

```
nv set vrf RED table auto
nv set bridge domain br_default vlan 100
nv set interface swp4 bridge domain br_default access 100
nv set interface vlan100 ipv4 vrr state enabled
nv set interface vlan100 type svi
nv set interface vlan100 ipv4 address 10.100.20.2/24
nv set interface vlan100 ipv4 vrr address 10.100.20.1/24
nv set interface vlan100 ipv4 vrr mac-address 00:00:00:00:2a:20
nv set interface swp51.100,vlan100 vrf RED
nv set interface swp51.100,vlan100 vlan 100
nv config apply
```

3. **[spine01]:** Repeat for **spine01**. A code snippet is provided below for easy copy and paste.

```
nv set vrf RED table auto
nv set interface swp1.100 base-interface swp1
nv set interface swp1.100,swp2.100 vrf RED
nv set interface swp1.100,swp2.100 type sub
nv set interface swp1.100,swp2.100 vlan 100
nv set interface swp2.100 base-interface swp2
nv config apply
```

VRF BGP Configuration

Let's configure BGP unnumbered peering for VRF RED.

1. **[leaf01]:** On **leaf01**, enable BGP.

```
cumulus@leaf01:mgmt:~$ nv set vrf RED router bgp address-family ipv4-unicast state enabled
cumulus@leaf01:mgmt:~$ nv set vrf RED router bgp address-family ipv4-unicast network 10.100.10.0/24
cumulus@leaf01:mgmt:~$ nv set vrf RED router bgp autonomous-system 65101
cumulus@leaf01:mgmt:~$ nv set vrf RED router bgp state enabled
cumulus@leaf01:mgmt:~$ nv set vrf RED router bgp neighbor swp51.100 remote-as external
cumulus@leaf01:mgmt:~$ nv set vrf RED router bgp neighbor swp51.100 type unnumbered
cumulus@leaf01:mgmt:~$ nv set vrf RED router bgp path-selection multipath aspath-ignore enabled
cumulus@leaf01:mgmt:~$ nv config apply
```

2. **[leaf02]:** Repeat for **leaf02**. A code snippet is provided below for easy copy and paste.

```
nv set vrf RED router bgp address-family ipv4-unicast state enabled
nv set vrf RED router bgp address-family ipv4-unicast network 10.100.20.0/24
nv set vrf RED router bgp autonomous-system 65102
nv set vrf RED router bgp state enabled
nv set vrf RED router bgp neighbor swp51.100 remote-as external
nv set vrf RED router bgp neighbor swp51.100 type unnumbered
nv set vrf RED router bgp path-selection multipath aspath-ignore enabled
nv config apply
```

3. **[spine01]:** Repeat for **spine01**. A code snippet is provided below for easy copy and paste.

```
nv set vrf RED router bgp autonomous-system 65201
nv set vrf RED router bgp state enabled
nv set vrf RED router bgp neighbor swp1.100 remote-as external
nv set vrf RED router bgp neighbor swp1.100 type unnumbered
nv set vrf RED router bgp neighbor swp2.100 remote-as external
nv set vrf RED router bgp neighbor swp2.100 type unnumbered
nv set vrf RED router bgp path-selection multipath aspath-ignore enabled
nv config apply
```

Verify Routing Table

Verify the routing table for both **default** and **RED** VRFs.

1. **[leaf01]:** On **leaf01**, verify.

```
cumulus@leaf01:mgmt:~$ nv show vrf default router rib ipv4 route

Flags - * - selected, q - queued, o - offloaded, i - installed, S - fib-selected, x - failed

Route           Protocol  Distance  Uptime   NHGId  Metric  Flags
-----
10.0.10.0/24     connected 0          0:32:31  14     1024    i
                 connected 0          0:32:31  15     0       *Si
10.0.10.1/32     local     0          0:32:31  14     0       *Si
10.0.10.2/32     local     0          0:32:31  15     0       *Si
10.0.20.0/24     bgp       20         0:03:25  26     0       *Si
10.255.255.1/32  connected 0          0:32:31  12     0       *Si
                 local     0          0:32:31  12     0       i
10.255.255.2/32  bgp       20         0:03:25  26     0       *Si
10.255.255.101/32 bgp       20         0:03:25  26     0       *Si
```

```
cumulus@leaf01:mgmt:~$ nv show vrf RED router rib ipv4 route

Flags - * - selected, q - queued, o - offloaded, i - installed, S - fib-selected, x - failed

Route           Protocol  Distance  Uptime   NHGId  Metric  Flags
-----
0.0.0.0/0        kernel   255        0:03:54  30     8192    *Si
10.100.10.0/24   connected 0          0:03:54  34     1024    i
                 connected 0          0:03:54  35     0       *Si
10.100.10.1/32   local     0          0:03:54  34     0       *Si
10.100.10.2/32   local     0          0:03:54  35     0       *Si
10.100.20.0/24   bgp       20         0:00:38  43     0       *Si
```

2. **[leaf02]:** On **leaf02**, verify.

```
cumulus@leaf02:mgmt:~$ nv show vrf default router rib ipv4 route
```

```
Flags - * - selected, q - queued, o - offloaded, i - installed, S - fib-selected, x - failed
```

Route	Protocol	Distance	Uptime	NHGI	Metric	Flags
10.0.10.0/24	bgp	20	0:06:49	26	0	*Si
10.0.20.0/24	connected	0	0:32:40	14	1024	i
	connected	0	0:32:40	15	0	*Si
10.0.20.1/32	local	0	0:32:40	14	0	*Si
10.0.20.3/32	local	0	0:32:40	15	0	*Si
10.255.255.1/32	bgp	20	0:06:49	26	0	*Si
10.255.255.2/32	connected	0	0:32:40	12	0	*Si
	local	0	0:32:40	12	0	i
10.255.255.101/32	bgp	20	0:06:49	26	0	*Si

```
cumulus@leaf02:mgmt:~$ nv show vrf RED router rib ipv4 route
```

```
Flags - * - selected, q - queued, o - offloaded, i - installed, S - fib-selected, x - failed
```

Route	Protocol	Distance	Uptime	NHGI	Metric	Flags
0.0.0.0/0	kernel	255	2025-08-20T08:50:59Z	22	8192	*Si
10.100.10.0/24	bgp	20	2025-08-20T08:55:35Z	48	0	*Si
10.100.20.0/24	connected	0	2025-08-20T08:51:00Z	34	1024	i
	connected	0	2025-08-20T08:51:00Z	35	0	*Si
10.100.20.1/32	local	0	2025-08-20T08:51:00Z	34	0	*Si
10.100.20.2/32	local	0	2025-08-20T08:51:00Z	35	0	*Si

3. **[spine01]:** On **spine01**, verify.

```
cumulus@spine01:mgmt:~$ nv show vrf default router rib ipv4 route
```

```
Flags - * - selected, q - queued, o - offloaded, i - installed, S - fib-selected, x - failed
```

Route	Protocol	Distance	Uptime	NHGI	Metric	Flags
10.0.10.0/24	bgp	20	0:04:09	18	0	*Si
10.0.20.0/24	bgp	20	0:04:09	20	0	*Si
10.255.255.1/32	bgp	20	0:04:09	18	0	*Si
10.255.255.2/32	bgp	20	0:04:09	20	0	*Si
10.255.255.101/32	connected	0	0:35:59	7	0	*Si
	local	0	0:35:59	7	0	i

```
cumulus@spine01:mgmt:~$ nv show vrf RED router rib ipv4 route
```

```
Flags - * - selected, q - queued, o - offloaded, i - installed, S - fib-selected, x - failed
```

Route	Protocol	Distance	Uptime	NHGI	Metric	Flags
0.0.0.0/0	kernel	255	2025-08-20T08:50:59Z	24	8192	*Si
10.100.10.0/24	bgp	20	2025-08-20T08:55:34Z	41	0	*Si
10.100.20.0/24	bgp	20	2025-08-20T08:55:34Z	40	0	*Si

Verify Connectivity

Verify our VRF is working by testing connectivity between the servers.

1. Verify end to end connectivity for servers for the VRF they belong to (intra-VRF).

```
ubuntu@server01:~$ ping 10.0.20.102 -c 3
```

```
PING 10.0.20.102 (10.0.20.102) 56(84) bytes of data.
```

```
64 bytes from 10.0.20.102: icmp_seq=1 ttl=61 time=1.84 ms
```

```
64 bytes from 10.0.20.102: icmp_seq=2 ttl=61 time=1.97 ms
```

```
64 bytes from 10.0.20.102: icmp_seq=3 ttl=61 time=2.06 ms
```

```
--- 10.0.20.102 ping statistics ---
```

```
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
```

```
rtt min/avg/max/mdev = 1.837/1.956/2.060/0.091 ms
```

```
ubuntu@server02:~$ ping 10.0.10.101 -c 3
```

```
PING 10.0.10.101 (10.0.10.101) 56(84) bytes of data.
```

```
64 bytes from 10.0.10.101: icmp_seq=1 ttl=61 time=1.56 ms
```

```
64 bytes from 10.0.10.101: icmp_seq=2 ttl=61 time=1.64 ms
```

```
64 bytes from 10.0.10.101: icmp_seq=3 ttl=61 time=1.64 ms
```

```
--- 10.0.10.101 ping statistics ---
```

```
3 packets transmitted, 3 received, 0% packet loss, time 1998ms
```

```
rtt min/avg/max/mdev = 1.562/1.612/1.638/0.035 ms
```

```
ubuntu@server03:~$ ping 10.100.20.104 -c 3
```

```
PING 10.100.20.104 (10.100.20.104) 56(84) bytes of data.
```

```
64 bytes from 10.100.20.104: icmp_seq=1 ttl=61 time=1.99 ms
```



```
64 bytes from 10.100.20.104: icmp_seq=2 ttl=61 time=1.87 ms
64 bytes from 10.100.20.104: icmp_seq=3 ttl=61 time=1.76 ms

--- 10.100.20.104 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 1.764/1.875/1.989/0.091 ms
```

```
ubuntu@server04:~$ ping 10.100.10.103 -c 3
PING 10.100.10.103 (10.100.10.103) 56(84) bytes of data.
64 bytes from 10.100.10.103: icmp_seq=1 ttl=61 time=2.80 ms
64 bytes from 10.100.10.103: icmp_seq=2 ttl=61 time=2.08 ms
64 bytes from 10.100.10.103: icmp_seq=3 ttl=61 time=2.12 ms

--- 10.100.10.103 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 2.075/2.331/2.800/0.331 ms
```

2. Verify prefixes are NOT leaked between VRFs (inter-VRF).

```
ubuntu@server01:~$ ping 10.100.20.104 -c 3
PING 10.100.20.104 (10.100.20.104) 56(84) bytes of data.
From 10.0.10.1 icmp_seq=1 Destination Net Unreachable
From 10.0.10.1 icmp_seq=2 Destination Net Unreachable
From 10.0.10.1 icmp_seq=3 Destination Net Unreachable

--- 10.100.20.104 ping statistics ---
3 packets transmitted, 0 received, +3 errors, 100% packet loss, time 2048ms
```

```
ubuntu@server02:~$ ping 10.100.20.104 -c 3
PING 10.100.20.104 (10.100.20.104) 56(84) bytes of data.
From 10.0.20.1 icmp_seq=1 Destination Net Unreachable
From 10.0.20.1 icmp_seq=2 Destination Net Unreachable
From 10.0.20.1 icmp_seq=3 Destination Net Unreachable

--- 10.100.20.104 ping statistics ---
3 packets transmitted, 0 received, +3 errors, 100% packet loss, time 2027ms
```

```
ubuntu@server03:~$ ping 10.0.10.101 -c 3
PING 10.0.10.101 (10.0.10.101) 56(84) bytes of data.
From 10.100.10.2 icmp_seq=1 Destination Host Unreachable
From 10.100.10.2 icmp_seq=2 Destination Host Unreachable
From 10.100.10.2 icmp_seq=3 Destination Host Unreachable

--- 10.0.10.101 ping statistics ---
3 packets transmitted, 0 received, +3 errors, 100% packet loss, time 2025ms
```

```
ubuntu@server04:~$ ping 10.0.20.102 -c 3
PING 10.0.20.102 (10.0.20.102) 56(84) bytes of data.
From 10.100.20.2 icmp_seq=1 Destination Host Unreachable
From 10.100.20.2 icmp_seq=2 Destination Host Unreachable
From 10.100.20.2 icmp_seq=3 Destination Host Unreachable

--- 10.0.20.102 ping statistics ---
3 packets transmitted, 0 received, +3 errors, 100% packet loss, time 2060ms
```

This concludes the NVIDIA Cumulus Linux Workshop.

Appendix A: Enable SSH In Your Lab

This appendix will show you how to enable SSH in your lab so you can use your preferred SSH client.

SSH Keys & Clients

An SSH client is a software application that establishes and manages secure connections to remote servers using the SSH protocol. An SSH key is a pair of cryptographic keys used for authentication in the SSH protocol. It consists of a public key and a private key.

Different systems are better optimized for different SSH clients. PuTTY is widely used for Windows machines, whereas Linux and macOS machines include built-in SSH clients via the terminal. There are lots of other clients available should you need, such as OpenSSH, iTerm2, and others.

Generate a Key

You must have an SSH private and public keypair. They often consist of two files called `id_rsa` and `id_rsa.pub` files but come in different variations depending on your client.

You can generate a new one, but you may already have one. Check in your home directory for a `.ssh` folder that contains `id_rsa` and `id_rsa.pub` files.

- > **Unix:** `~/.ssh/`
- > **Windows:** `C:/Users/user/.ssh`

```
laptop:~/user$ cat ~/.ssh/id_rsa.pub
ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQAC7feqSFSAUxpe2qTv77+pEk82C/i4A1XVcOQ15tWBCAq1tPWmZHJCPcElFTjeG2wMY
Mx2Kmb3kYwrLcwFTk06avziBhjMwIprFiupWCKykRPM4IOHkiWDS/htpZfBdwuIFXV4MQtCiD9zUhlI0Uq0Is+1VtE1Q0/38
N7sSa7FHaVNpDpJ0Qf3PYVdfhk/BG19WQ1yKMUSj0aRrAHUIckiQs2H5Wm198ciKkg14AxoDM9QB+f1cC13We52ei5tWqV8C
gLehhrdjEXn+iXNdkcg+nGka1syUSYntotally+fake+key+MkEFwD9v16SmJYDK67w5RaHTjBS52UoRjnEEN
user@hostname.local
```

If you do not have a key, or wish to generate a new one, the links below are suggestions only for generating ssh keys. The information on these pages is not owned or controlled by NVIDIA. Use with caution. Often the ssh *private* key is password protected. You should know the password for your key. NVIDIA does not know the password/passphrase of your ssh private key. If you do not know or have forgotten, you may need to generate a new set of keys.

If on Windows, PuTTY requires private keys in its own format and does not use an `.pub` file directly. You **must** use a `.ppk` file as a private key. You can generate or convert one using PuTTYgen.

- > **Windows (using PuTTY):** <https://www.ssh.com/ssh/putty/windows/puttygen>
- > **Mac:** <https://docs.joyent.com/public-cloud/getting-started/ssh-keys/generating-an-ssh-key-manually/manually-generating-your-ssh-key-in-mac-os-x>
- > **Ubuntu/linux:** <https://help.ubuntu.com/community/SSH/OpenSSH/Keys>

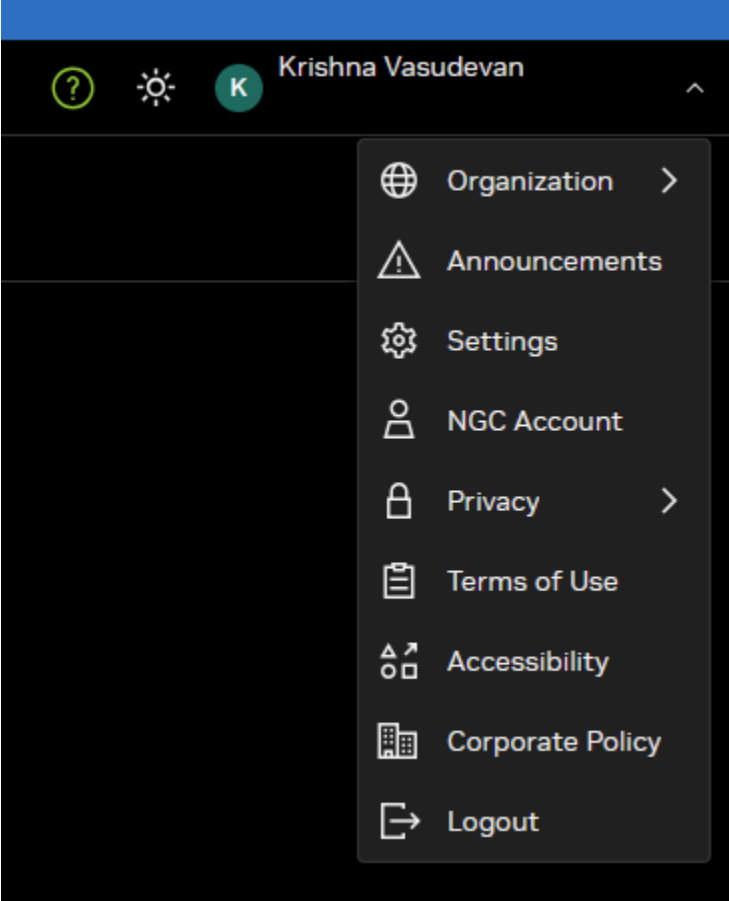
Your goal is to have an SSH keypair and access to your **public key** string that looks like the key string below.

```
ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQAC7feqSFSAUxpe2qTv77+pEk82C/i4A1XVcOQ15tWBCAq1tPWmZHJCPcElFTjeG2wMY
Mx2Kmb3kYwrLcwFTk06avziBhjMwIprFiupWCKykRPM4IOHkiWDS/htpZfBdwuIFXV4MQtCiD9zUhlI0Uq0Is+1VtE1Q0/38
N7sSa7FHaVNpDpJ0Qf3PYVdfhk/BG19WQ1yKMUSj0aRrAHUIckiQs2H5Wm198ciKkg14AxoDM9QB+f1cC13We52ei5tWqV8C
gLehhrdjEXn+iXNdkcg+nGka1syUSYntotally+fake+key+MkEFwD9v16SmJYDK67w5RaHTjBS52UoRjnEEN
user@hostname.local
```

Warning: Please be careful. We can't know in advance if you have existing ssh keys and if you do, if they are used by anything on your machine or for any important tools in your workflow or line of work. There is risk that *regenerating default keys may break something* you do or some other remote logins that depend on your default/existing keys. If you already have default keys, please use them and do not regenerate them.

Copying your Key into Air

1. In Air, click your account name in the top right and then click **Settings**.



2. Copy your public key into the **Public Key** area and give it a name.

SSH Key

Keys make it easy to securely connect to and manage your simulation.
Any SSH keys you add here will be automatically included in your simulation's list of authorized keys when it is created.

Name *

Name

Public Key *

Enter your public SSH key here (commonly located at ~/.ssh/id_rsa.pub). Do not include your private key.

Verify Keys

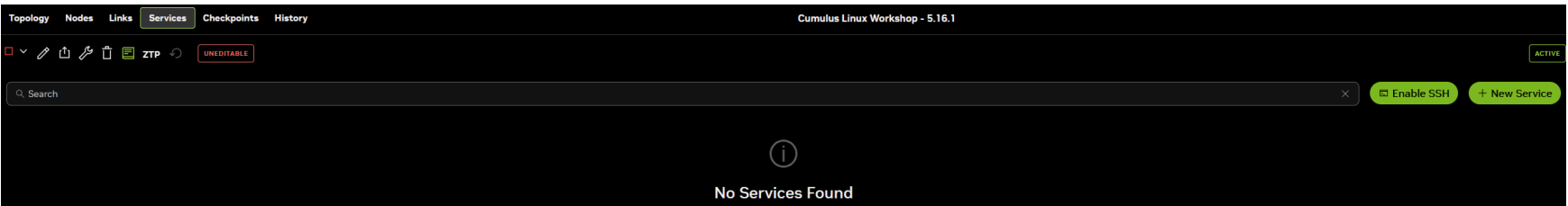
You can verify your keys were inserted into any *newly created* simulations.

1. In the `oob-mgmt-server`, check the `~/.ssh/authorized_keys` file for your key.

```
ubuntu@oob-mgmt-server:~/.ssh$ cat authorized_keys
ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQ7feqSFSAUxpe2qTv77+pEk82C/i4AlXVcOQl5tWBCAq1tPWmZHZJCPcElFTjeG2wMYMx2Kmb3k
YwrLcwFtk06avziBhjMwIprFiupWCkykRPM4IOHkiWDS/htpZfBdwuIFXV4MQtCiD9zUhLi0Uq0Is+1VtE1Q0/38N7sSa7FHaVNpDpJO
Qf3PYVdfhk/BG19WQlyKMUSj0aRrAHUIckiQs2H5Wm198ciKkg14AxoDM9QB+f1cCl3We52ei5tWqV8CgLehhrdjEXn+iXNdkcg+nGka
1syUSYntotally+fake+key+MkEFwD9v16SmJYDK67w5RaHTjBS52UoRjnEEN rsa-key-20240411
```

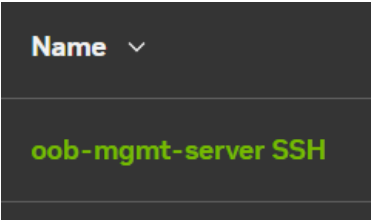
Enable SSH

1. Within your simulation, click the **Services** → **Enable SSH** button to enable the SSH service for the sim. **This exposes the `oob-mgmt-server` to the Internet.**



Name	Node	Interface	Service Port	Type	External Host	External Port	Actions
oob-mgmt-server SSH	oob-mgmt-server	eth0	22	SSH	ngc-worker96.air-inside.nvidia.com	18907	

2. **If your web browser is configured with an application to handle SSH URLs**, you can click on the hyperlink for the SSH service to automatically launch the application to handle the SSH connection and connect with the correct username, IP address, and port number.



If your browser is *not* configured to handle SSH URLs, manually input the information into your preferred SSH client.

- > **Windows:** Download [PuTTY](#) and optionally [WinSCP](#)
- > **Mac/Linux:** Use a terminal/shell

Note: This SSH connection does not use the default destination TCP port 22. Ensure that the external port is specified in your SSH client.

Note: If you are prompted for a password when connecting to SSH, it is the password for your SSH private key. This is not a password being requested from the server for authentication against the ubuntu user in `oob-mgmt-server`.

Linux/Mac OS using Terminal

Using the `ssh` built-in client, input the SSH information from Air into a terminal.

```
user@laptop$ ssh ubuntu@f8c4f4d9.workers.ngc.air.nvidia.com -p 27573
The authenticity of host '[f8c4f4d9.workers.ngc.air.nvidia.com]:27573 ([x.x.x.x]:27573)' can't
be established.
ECDSA key fingerprint is SHA256:XXXXXX.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '[f8c4f4d9.workers.ngc.air.nvidia.com]:27573,[x.x.x.x]:27573 (ECDSA)
to the list of known hosts.
Enter passphrase for key '/Users/user/.ssh/id_rsa': <your ssh private key passphrase>
Welcome to Ubuntu 22.04.2 LTS (GNU/Linux 5.15.0-67-generic x86_64)

<banner omitted for brevity>

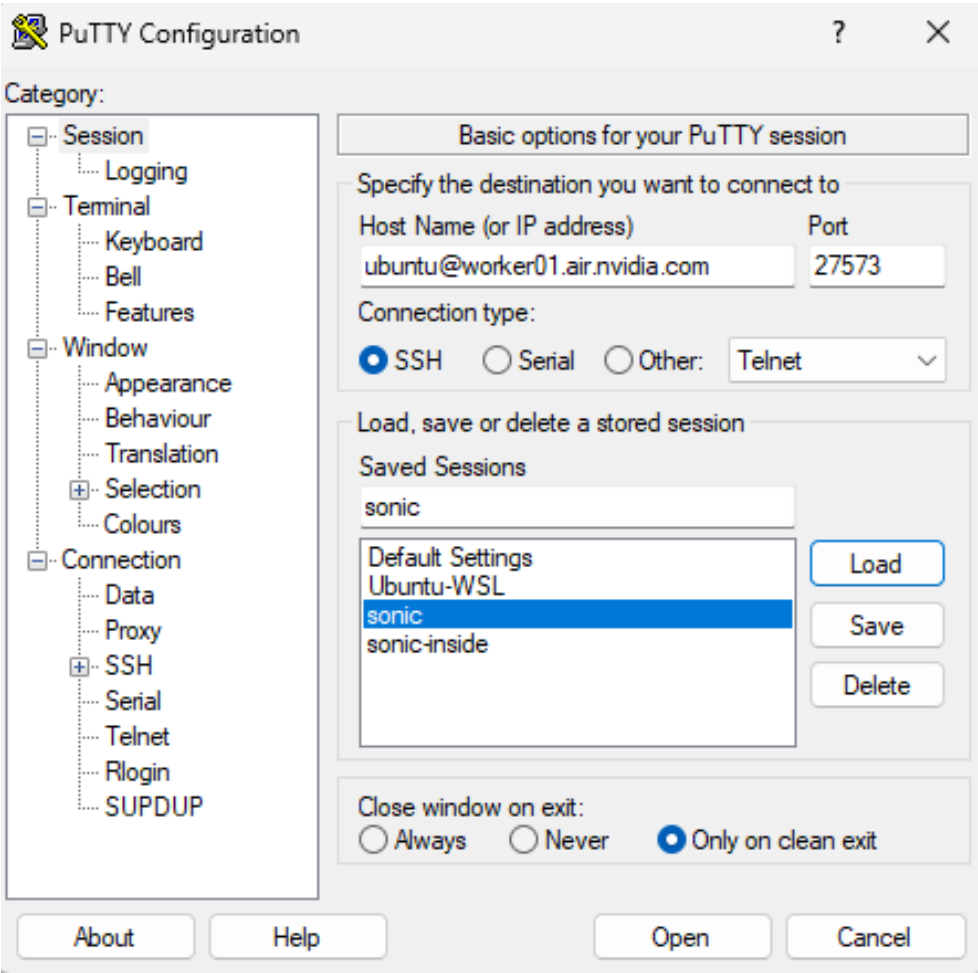
ubuntu@oob-mgmt-server:~$
```

Windows using PuTTY

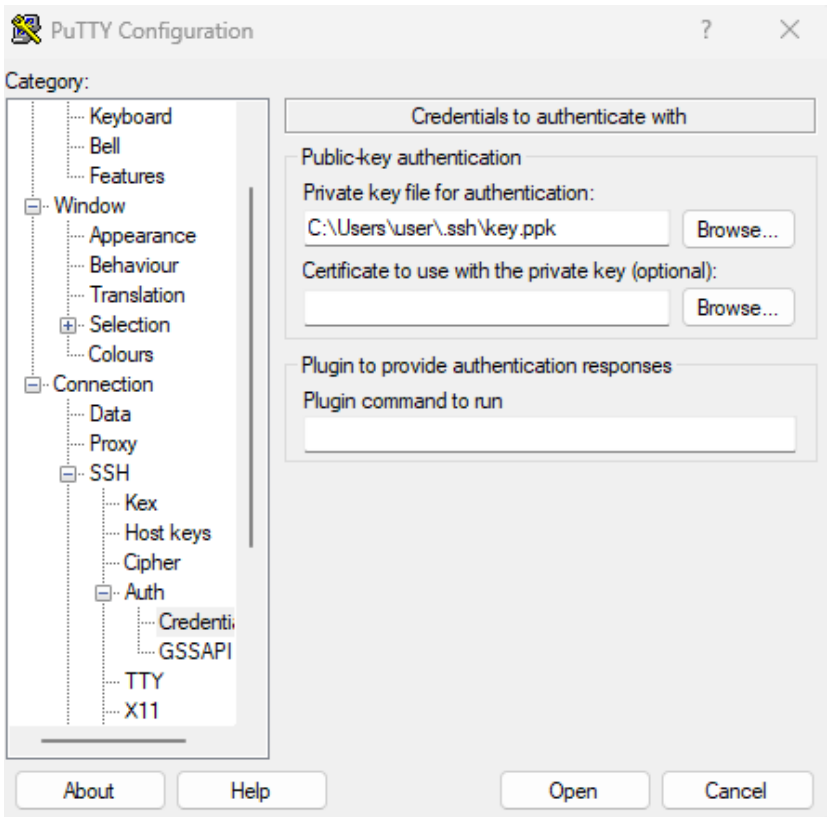
PuTTY is widely used on Windows for SSH connection and other functions.

PuTTY requires a special `.ppk` private key format for authentication. As discussed in [Generate a Key](#), you can generate a new private key with PuTTYgen. Find more information [here](#).

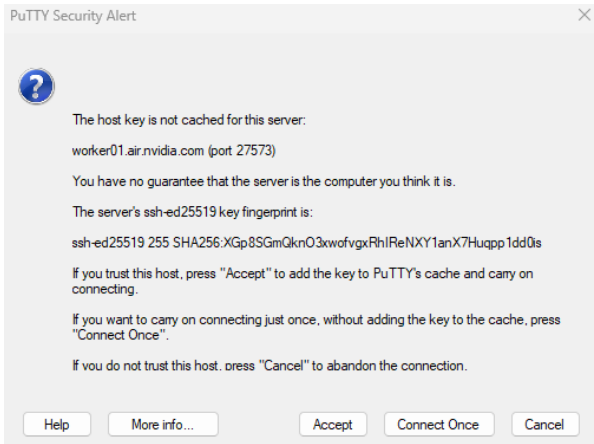
1. Input the SSH information from Air into PuTTY.



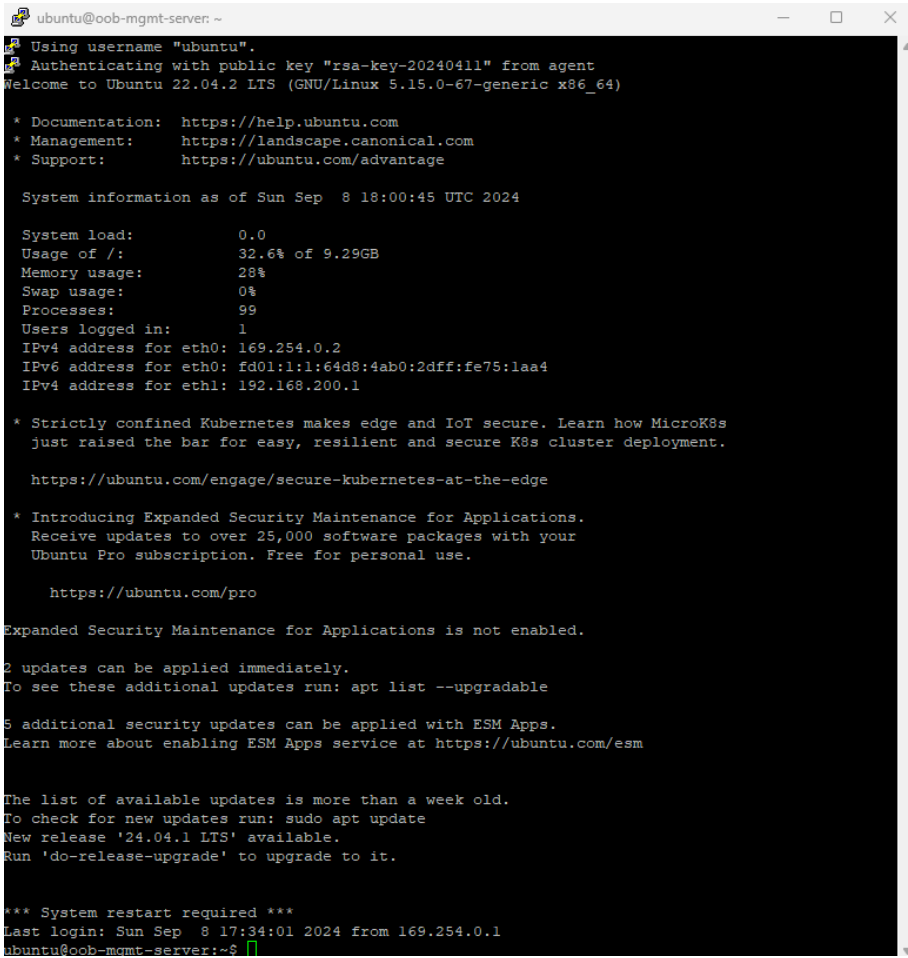
2. Navigate to **Connection** → **SSH** → **Auth** → **Credentials**. Input your **.ppk private key** into the **Private key file for authentication** box.



3. Click **Open**.
4. Click **Accept** or **Connect Once**.



5. Login with username **ubuntu**.
6. If prompted, enter your password for your SSH private key if you created one.
7. You are now logged into **oob-mgmt-server** with PuTTY.



WinSCP

Optionally, you can use WinSCP for easy file management between your local system and Air. It also provides a graphical interface. WinSCP can be configured to use PuTTY's SSH client for secure connections, combining the file transfer capabilities of WinSCP with the terminal capabilities of PuTTY.

About NVIDIA (Cumulus Networks was acquired by NVIDIA in June 2020)

NVIDIA is leading the transformation of bringing web-scale networking to enterprise cloud. Its network switch operating system, NVIDIA © Cumulus Linux, is the only solution that allows you to affordably build and efficiently operate your network like the world's largest data center operators, unlocking vertical network stacks. By allowing operators to use standard hardware components, NVIDIA Cumulus Linux offers unprecedented operational speed and agility, at the industry's most competitive cost. For more information visit <https://www.nvidia.com/en-us/networking/ethernet-switching/>.

© 2025 NVIDIA Corporation. All rights reserved. NVIDIA, the NVIDIA logo, and NVIDIA © Cumulus Linux, NVIDIA © Cumulus NetQ are trademarks and/or registered trademarks of NVIDIA Corporation in the U.S. and other countries. Other company and product names may be trademarks of the respective companies with which they are associated.